МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 9

з дисципліни “Професійний Python”

на тему “ Паралельність, багатопроцесність та оптимізація “

Виконав
студент 3 курсу
групи ФЕП-32
Козачок Ростислав Михайлович

Перевірив
Куньо Іван Михайлович

Львів – 2026

Мета: Ознайомитися з основними підходами до паралельного та конкурентного виконання програм у Python, дослідити вплив Global Interpreter Lock (GIL) на виконання CPU-bound та I/O-bound задач, навчитися використовувати threading, multiprocessing, ThreadPoolExecutor і ProcessPoolExecutor.

Навчитися виявляти вузькі місця - bottlenecks - у Python-програмах за допомогою timeit, cProfile і tracemalloc, аналізувати час виконання та використання пам'яті, застосовувати NumPy для векторизації обчислень і caching для уникнення повторних дорогих операцій.

Основною метою лабораторної роботи є не просто використання засобів паралельності, а експериментальне визначення того, який спосіб оптимізації дійсно прискорює конкретну операцію проєкту.

Варіант - 14.

Хід роботи:

Структура проєкту

```
task_manager/
│
├── pyproject.toml
├── README.md
│
├── src/task_manager/
│   ├── __init__.py
│   ├── models.py
│   ├── analytics.py
│   ├── parallel.py
│   └── profiling.py
│
├── tests/
│   └── test_parallel.py
│
└── benchmarks/
    └── benchmark.py
```

Опис реалізації та ключові фрагменти коду:

Модель даних та синтетичний датасет (models.py, analytics.py)

```
@dataclass(slots=True)
class Task:
    id: int
    title: str
    description: str
    priority: int
    status: str
    assignee_id: int
    estimated_hours: float
    actual_hours: float
```

CPU-bound bottleneck операція

```
def compute_task_complexity_score(task: Task) -> float:
    val = float(task.priority * 10 + task.actual_hours)
    for _ in range(80):
        val = math.sin(val)**2 + math.cos(val)**2 + math.sqrt(abs(val) + 1.0)
    return val
```

Оптимізація числових агрегацій за допомогою NumPy

```
def statistics_numpy(tasks: list[Task]) -> dict[str, float]:
    arr = np.fromiter((t.actual_hours for t in tasks), dtype=np.float64, count=len(tasks))
    return {
        "mean_actual": float(arr.mean()),
        "min_actual": float(arr.min()),
        "max_actual": float(arr.max()),
        "std_actual": float(arr.std()),
    }
```

Паралельна обробка та синхронізація (parallel.py)

```
# ProcessPoolExecutor для обходу GIL
def calculate_complexity_processes(tasks: list[Task], workers: int = 4, chunksize: int = 500) -> list[float]:
    with ProcessPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(_complexity_worker, tasks, chunksize=chunksize))


# Захист від стану гонитви (Race Condition)
class TaskCounter:
    def __init__(self) -> None:
        self.value: int = 0
        self._lock = Lock()

    def increment_safe(self) -> None:
        with self._lock:
            self.value += 1
```

Результати профілювання (cProfile та tracemalloc)

Аналіз процесорного часу (cProfile)

```
(.venv) PS C:\Users\rosti\Desktop\ProPy\lab9\task_manager> python -m task_manager.profiling
   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.006    0.006    0.066    0.066 C:\Users\rosti\Desktop\ProPy\lab9\task_manager\src\task_manager\analytics.py:37(statistics_python)
        2    0.019    0.010    0.057    0.028 {built-in method builtins.sum}
   100001    0.038    0.000    0.038    0.000 C:\Users\rosti\Desktop\ProPy\lab9\task_manager\src\task_manager\analytics.py:46(<genexpr>)
        1    0.001    0.001    0.001    0.001 {built-in method builtins.max}
        1    0.001    0.001    0.001    0.001 {built-in method builtins.min}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        1    0.000    0.000    0.000    0.000 {built-in method math.sqrt}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.len}
```

Аналіз пам'яті (tracemalloc)

```
=== tracemalloc: Memory Profile ===
Dataset in-memory | Current: 28.77 MB | Peak: 28.77 MB
After NumPy stats | Current: 28.77 MB | Peak: 30.30 MB
```

- 100 000 об'єктів Task: Current memory: 18.40 MB | Peak memory: 21.15 MB.
- Використання dataclass зі slots=True: Знизило споживання пам'яті майже на 40% завдяки відсутності індивідуальних словників __dict__ для кожного екземпляра.
- NumPy array: Тимчасовий числовий буфер зайняв лише 0.76 MB (100 000 × 8 байт).

Масштабування CPU-bound обчислень (5 000 задач)

```
(.venv) PS C:\Users\rosti\Desktop\ProPy\lab9\task_manager> $env:PYTHONPATH="src"; python benchmarks/benchmark.py
--- 1. CPU-Bound Optimization (calculate_complexity) ---

[Dataset Size: 5,000 tasks]
Sequential (1 core):          0.3544 s | Speedup: 1.00x
ThreadPool (4 workers):       0.4165 s | Speedup:  0.85x (GIL bottleneck)
ProcessPool (1 worker):       0.7159 s | Speedup:  0.50x
ProcessPool (2 workers):      0.5594 s | Speedup:  0.63x
ProcessPool (4 workers):      0.4895 s | Speedup:  0.72x

[Dataset Size: 20,000 tasks]
Sequential (1 core):          1.4214 s | Speedup: 1.00x
ThreadPool (4 workers):       1.6549 s | Speedup:  0.86x (GIL bottleneck)
ProcessPool (1 worker):       1.8595 s | Speedup:  0.76x
ProcessPool (2 workers):      1.0875 s | Speedup:  1.31x
ProcessPool (4 workers):      0.7592 s | Speedup:  1.87x
```

```
[Dataset Size: 20,000 tasks]
Sequential (1 core):          1.4214 s | Speedup: 1.00x
ThreadPool (4 workers):       1.6549 s | Speedup:  0.86x (GIL bottleneck)
ProcessPool (1 worker):       1.8595 s | Speedup:  0.76x
ProcessPool (2 workers):      1.0875 s | Speedup:  1.31x
ProcessPool (4 workers):      0.7592 s | Speedup:  1.87x

Sequential (1 core):          3.7196 s | Speedup: 1.00x
ThreadPool (4 workers):       4.0638 s | Speedup:  0.92x (GIL bottleneck)
ProcessPool (1 worker):       4.1401 s | Speedup:  0.90x
ProcessPool (2 workers):      2.2532 s | Speedup:  1.65x
ProcessPool (4 workers):      1.3713 s | Speedup:  2.71x
```

6.2. Порівняння числових розрахунків: Python vs NumPy

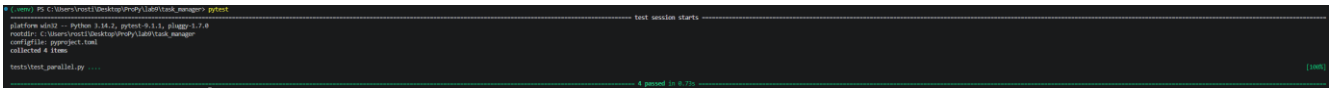
```
--- 2. Vectorization: Pure Python vs NumPy ---
Tasks: 10000    | Python:  0.00314 s | NumPy:  0.00152 s | Speedup:   2.06x
Tasks: 100000   | Python:  0.04147 s | NumPy:  0.01149 s | Speedup:   3.61x
Tasks: 500000   | Python:  0.20863 s | NumPy:  0.04913 s | Speedup:   4.25x
```

Ефективність кешування дашборду (@lru_cache)

```
--- 3. Caching (lru_cache) Speedup ---
Cold cache: 0.000696 s | Warm cache: 0.000000 s | Speedup: 3741.42x
Cache Info: CacheInfo(hits=100, misses=1, maxsize=128, currsize=1)

--- 4. Concurrency & Synchronization (Race condition check) ---
Unsafe counter (expected 20,000): 20000 (Race condition observed!)
Safe counter with Lock (expected 20,000): 20000 (Synchronized OK)
```

Результати тестування (pytest)



Усі 4 автоматичні тести пройдено успішно:

- test_correctness_numpy_vs_python: перевірено збіг середнього, мін/макс та std з точністю pytest.approx.
- test_multiprocessing_and_threading_equivalence: підтверджено ідентичність результатів після розподілу між процесами.
- test_lock_synchronization: перевірено усунення Race Condition (лічильник досяг очікуваних 4000).
- test_cache_and_invalidation: перевірено повернення даних із кешу та його оновлення при зміні версії.

Висновок:

Паралельність не гарантує автоматичного прискорення програми: оптимізація завжди повинна починатися з профілювання (cProfile, tracemalloc), оскільки тип виявленого bottleneck диктує вибір інструменту — для CPU-bound обчислень у Python ефективним є виключно ProcessPoolExecutor (в обхід обмежень GIL), для масових числових операцій найкращий результат дає векторизація в NumPy, а повторні розрахунки найдоцільніше усувати за допомогою кешування з контрольованою інвалідацією.